

# The Hippie Scientist

## Deep Codebase & Product Audit

thehippiescientist.net · March 2026

---

1. Executive Summary
2. Top 10 Problems
3. Highest-Leverage Opportunities
4. Codex Prompt Plan (20 prompts)
5. Recommended Prompt Order
6. PR Review Checklist
7. Optional Stretch Prompts

## 1. Executive Summary

---

### What this site is trying to be

A science-first, harm-reduction-framed reference for psychoactive herbs and plant compounds — targeting curious readers who want more rigor than typical wellness content and less judgment than mainstream drug-safety sources. The brand positioning ('Hippie Scientist') is strong and differentiated. The data infrastructure ambition is genuinely impressive.

### What it does well

- The data pipeline is architecturally sound and mature: normalization layer, publication gating, enrichment workflows, quality-gate scripts, and structured data generation all exist and show real engineering discipline.
- The interactive tools (Blend Builder, Interaction Checker, Effect Search) are genuinely differentiated. Nobody else in this niche has them.
- The dark-mode aura/nebula aesthetic is cohesive and brand-reinforcing.
- BrowserRouter is correctly in use; prerendering and sitemap generation are hooked into the build pipeline; JSON-LD exists.
- The affiliate UX — confidence-gated, disclosure-included, review-status-tracked — is well-thought-out and FTC-defensible.

## What is undermining it most

- **The database is almost entirely unindexable.** Only 12 of 812 herbs and 0 of 394 compounds pass the publication gate. Google cannot crawl meaningful metadata for ~98% of entity pages.
- **The blog has 60 draft 'daily note' posts with placeholder text** ('Placeholder reference A/B') and templated bodies — produced by a generator that makes no LLM call.
- **109 herb records have `name: 'nan'`** — a pandas/CSV NaN artifact baked into production data.
- **0 compound pages are indexable** — every one of 394 compounds fails quality gates on description completeness.
- **The About page uses a placeholder name ('Ari')** that doesn't match the actual operator.
- **No Vite manual chunking** — a likely 800KB+ initial JS bundle hurting Core Web Vitals.

## Overall verdict

**Potential: High.** The tooling, architecture intent, niche clarity, and interactive feature set are legitimately above average for indie content sites. The main blocker is data quality — the pipeline exists to fix it but has not been run to completion. If the publication gate can be cleared for even 100–200 herbs and 50 compounds, this becomes a crawlable, rankable entity database. That is the unlock.

## 2. Top 10 Problems

**P 1** 98% of herb pages and 100% of compound pages are not being prerendered/indexed ● **CRITICAL**

**WHY IT MATTERS** These are your core SEO assets. A herb/compound database where Google can only access 12 pages is functionally not a database from a search standpoint.

**IMPACT** Zero organic traffic to entity pages. All the effort building herb profiles is invisible to search engines.

**ROOT CAUSE** publication-manifest.json controls what gets prerendered. The manifest is generated by scripts/generate-indexable-herbs.mjs, which runs quality gates requiring: description  $\geq 60$  chars,  $\geq 2$  sources,  $\geq 1$  effect, completeness score  $\geq 10$ . 651 herbs fail on weakDescription, 667 on placeholderText. The data enrichment pipeline (enrichment:run) exists to fix this but has not been run to sufficient coverage.

**P 2** 109 herb records have name: 'nan' and slug: 'nan' ● **CRITICAL**

**WHY IT MATTERS** NaN artifacts from a pandas CSV import where the name column had missing values. They produce garbage URLs (/herbs/nan), pollute the data, and some slip through quality thresholds.

**IMPACT** Potential broken herb pages at /herbs/nan if any slip through the publication gate. Active data integrity issue across the entire pipeline.

**ROOT CAUSE** The CSV-to-JSON conversion script (scripts/convert-herbs-csv.ts / data:enrich) does not sanitize NaN string values from pandas export.

**P 3** 0 compound pages are indexable ● **CRITICAL**

**WHY IT MATTERS** Compound detail pages are a second major content category. Every single one of 394 compounds fails weakDescription with empty description fields.

**IMPACT** An entire content category that Google cannot crawl or index. Compounds are a key search intent (e.g., 'nuciferine effects', '5-HTP mechanism').

**ROOT CAUSE** The compound data was never enriched. The enrichment workflow targets herbs; compounds were bootstrapped structurally but never had descriptions written in.

**P 4** Daily-post generator creates template-only content with placeholder references ● **HIGH**

**WHY IT MATTERS**

60 files in content/blog/ with draft: true and body text: 'Placeholder reference A / Placeholder reference B'. If the draft filter breaks, 60 pages of content-free filler go live.

**IMPACT**

Content quality liability. If any are accidentally published, they damage E-E-A-T. The daily-post generator never calls an LLM API — it produces static templates.

**ROOT CAUSE**

scripts/generate-daily-post.mjs uses a static Markdown template with no API call. The body hardcodes 'Placeholder reference A/B' as sources.

**P**  
**5** **About page has wrong author name ('Ari')**

● **HIGH**

**WHY IT MATTERS**

The About page says 'I'm Ari' — a credibility-destroying inconsistency if the actual operator is Will (or anyone else).

**IMPACT**

Anyone who reads About carefully sees a name that doesn't match the site's social presence. Trust signal actively undermined.

**ROOT CAUSE**

Placeholder/old name left in src/pages/About.tsx.

**P**  
**6** **No Vite manual chunk splitting**

● **HIGH**

**WHY IT MATTERS**

framer-motion, react-force-graph-2d, three.js, @tsparticles, jspdf, and a large data layer all land in a single vendor bundle without manualChunks.

**IMPACT**

Likely 800KB+ initial JS bundle, hurting Core Web Vitals (LCP, INP) and mobile load time — directly correlated with search ranking.

**ROOT CAUSE**

vite.config.ts has no build.rollupOptions at all.

**P**  
**7** **Deploy workflow never runs the actual build**

● **HIGH**

**WHY IT MATTERS**

deploy.yml runs enrichment:release:gate and verify:redirects then calls the Netlify webhook — but never runs npm run build. A broken build is only caught by Netlify after the hook fires.

**IMPACT**

GitHub Actions shows green while Netlify fails. No pre-deploy validation of the compiled output.

**ROOT CAUSE**

The GH Actions deploy job is a webhook trigger, not a build runner. There is no npm run build step in the workflow.

**P**  
**8** **Disclaimer page has no affiliate/FTC disclosure**

● **MEDIUM**

**WHY IT MATTERS**

DisclaimerPage.tsx is purely a medical disclaimer. The affiliate disclosure lives only inline in CuratedProductModule. FTC guidelines require a clear, accessible disclosure.

**IMPACT**

Regulatory exposure. The inline disclosure in the product module is good but a standalone legal page should also mention it.

**ROOT CAUSE**

Disclaimer page content was written before the affiliate feature was built.

**P9** **ESLint has nearly all safety rules disabled**

● MEDIUM

**WHY IT MATTERS**

@typescript-eslint/no-unused-vars: 'off', no-explicit-any: 'off', react-hooks/exhaustive-deps: 'off'. These are the rules that catch real bugs in production.

**IMPACT**

The lint gate passes by construction. Stale deps in useEffect hooks, any-typed data flowing through UI, unused imports — all silent.

**ROOT CAUSE**

Rules were disabled to clear the lint gate without fixing the violations.

**P10** **Homepage has no single primary CTA — sections compete**

● MEDIUM

**WHY IT MATTERS**

8+ homepage sections with no clear conversion hierarchy. A new visitor cannot immediately identify this site's signature action.

**IMPACT**

Lower conversion to email capture, lower depth-per-session, weaker brand recall.

**ROOT CAUSE**

Homepage grew organically by section accretion rather than conversion-led design.

### 3. Highest-Leverage Opportunities

---

#### 1 Fix NaN herb artifacts and run enrichment to clear the publication gate for 100–200 herbs

This is the single change that goes from 12 indexed herb pages to 200+ indexed herb pages and unlocks meaningful organic traffic. The pipeline already exists — it needs to run to completion.

---

#### 2 Write minimal viable descriptions for the top 50 compounds

Even 2–3 sentence pharmacology summaries get the compound database above zero indexed pages. Compounds are a high-intent search category ('nuciferine effects', 'kava kavalactones mechanism').

---

#### 3 Add Vite manual chunk splitting

Separate framer-motion, @tsparticles, react-force-graph-2d, and jspdf into lazy vendor chunks. One-day change with significant LCP improvement. react-force-graph-2d alone is ~400KB and only used on /graph.

---

#### 4 Fix the About page author name

One-line change. Outsized trust signal impact. A credibility page with the wrong author name actively undermines every other trust signal on the site.

---

#### 5 Convert the daily-note blog generator to call a real LLM API

The pipeline infrastructure is there. The CI workflow is there. The content generation step just needs the API call. Right now you have a content moat with no content in it. Switching to real LLM-generated posts turns 60 drafts into 60 publishable entries.

---

#### 6 Add BreadcrumbList JSON-LD to herb and compound pages

Quick SERP appearance improvement. Breadcrumb rich results increase CTR from search. The breadcrumbJsonLd function appears to already exist in seo.ts — this may be a two-line fix.

---

## 4. Codex Prompt Plan

20 tightly-scoped, independently-reviewable prompts. Each is safe to run one at a time and produces a PR small enough to evaluate manually. Follow the wave order in Section 5.

### Prompt 1      **Scrub NaN herb records from production data**

#### GOAL

Remove all herb records where `name === 'nan'` (case-insensitive) or `slug === 'nan'` from production data and prevent them from re-entering via the data pipeline.

#### READ FIRST

`public/data/herbs_combined_updated.json`  
`scripts/convert-herbs-csv.ts`  
`scripts/quality-gate-data.mjs`  
`scripts/generate-indexable-herbs.mjs`

#### TASKS

1. In `scripts/convert-herbs-csv.ts`, add a sanitization step that drops any record where the resolved name matches `/^nan$/i` or is empty after stripping whitespace. Also drop records where slug resolves to 'nan'.
2. Write `scripts/verify-no-nan-records.mjs` that reads both herbs and compounds JSON files, counts records with `name/slug === 'nan'`, and exits 1 if any are found.
3. Add `node scripts/verify-no-nan-records.mjs` to the postbuild npm script.
4. Run the script against current data and report findings.

#### CONSTRAINTS

- Fix must be in the conversion/normalization step — not a one-time manual delete.
- Do NOT drop records whose name merely contains the substring 'nan' (e.g. 'Kava-Kava'). Only exact `/^nan$/i` matches.
- No new npm dependencies.

#### ACCEPTANCE CRITERIA

- ✓ `node scripts/verify-no-nan-records.mjs` exits 0.
- ✓ Re-running `npm run data:enrich` followed by the verify script continues to pass.
- ✓ Show: files changed, diffs, verification output.

# Raise herb publication gate pass rate

## GOAL

Identify herbs closest to passing the publication gate and auto-fill descriptions from existing fields (mechanism + effects + latin name) to get indexable herb count from 12 to at least 50.

## READ FIRST

public/data/publication-manifest.json (summaries.herbs.excludedByReason)  
scripts/generate-indexable-herbs.mjs (gate logic)  
public/data/herbs\_combined\_updated.json  
src/lib/data.ts (normalization contract)

## TASKS

1. Write scripts/report-description-candidates.mjs that identifies herbs failing ONLY on weakDescription and checks if description can be auto-generated from mechanism + effects + latin name. Output: public/data/description-candidates.json.
2. Write scripts/autofill-descriptions.mjs that constructs minimal descriptions: '{latin} is a {class} herb known for {effects[0..2]}. {mechanism}.' (>60 chars, no NaN tokens). Output: public/data/herbs-description-patches.json.
3. Write scripts/apply-description-patches.mjs that merges patches into herbs\_combined\_updated.json, only overwriting descriptions that are empty or <60 chars. Creates backup first.
4. After applying patches, re-run generate-indexable-herbs.mjs and report new indexable count.

## CONSTRAINTS

- Generated descriptions must pass NAN\_TOKEN\_PATTERN check from quality-gate-data.mjs.
- Do not fabricate pharmacological claims — only use data already in the record.
- The patch/apply pattern preserves rollback ability. All scripts must be idempotent.

## ACCEPTANCE CRITERIA

- ✓ Indexable herb count increases from 12 to at least 50.
- ✓ No generated description contains 'nan' as a word.
- ✓ Show: count before/after, 5 sample descriptions, failures.



Prompt 3

## Populate compound descriptions — get compound pages indexable

### GOAL

Every compound in `compounds_combined_updated.json` has an empty description. Construct minimal viable descriptions from existing fields so that at least 20 compounds pass the publication gate.

### READ FIRST

`public/data/compounds_combined_updated.json` (sample 20 records to understand available fields)

`public/data/publication-manifest.json` (`summaries.compounds.excludedByReason`)

`scripts/generate-indexable-compounds.mjs`

`src/lib/compound-data.ts`

### TASKS

1. Sample 20 compound records and print available fields to understand what data exists.
2. Write `scripts/autofill-compound-descriptions.mjs` that constructs descriptions from category + effects + mechanism ( $\geq 60$  chars, no NaN, skip IUPAC-only names). Output: `public/data/compounds-description-patches.json`.
3. Write `scripts/apply-compound-description-patches.mjs` that merges patches with backup.
4. Re-run `generate-indexable-compounds.mjs` and report new indexable count.

### CONSTRAINTS

- Same NaN-safety rules as Prompt 2.
- Prioritize compounds with recognizable common names over raw IUPAC-only slugs.
- Do not fabricate mechanism claims.

### ACCEPTANCE CRITERIA

- ✓ Indexable compound count increases from 0 to at least 20.
- ✓ `node scripts/verify-no-nan-records.mjs` continues to pass.
- ✓ Show: sample descriptions, count before/after, compounds skipped and why.

Prompt 4

## Fix About page author name and strengthen trust content

### GOAL

The About page contains 'I'm Ari' — a wrong/placeholder author name. Fix it and lightly strengthen trust signals without a full redesign.

### READ FIRST

`src/pages/About.tsx` (full file)

### TASKS

1. Replace 'I'm Ari' with correct attribution. If operator name should be removed entirely, use: 'I'm the researcher and writer behind Hippie Scientist' until explicitly specified.
2. Add a short paragraph under 'Methodology and editorial standard' section linking to `/methodology` using `<Link to="/methodology">`.

3. Add a short 'Affiliate disclosure' subsection at the bottom noting qualifying purchase commissions. Link to /disclaimer.

4. Correct Meta title from 'About Hippie Scientist' to 'About | The Hippie Scientist' (the 'The' was dropped).

#### CONSTRAINTS

- No new dependencies. Copy changes only — do not change layout/structure.
- Do not add photos, social links, or other elements unless already in codebase.

#### ACCEPTANCE CRITERIA

- ✓ String 'Ari' does not appear anywhere in src/pages/About.tsx.
- ✓ Page links to /methodology and /disclaimer.
- ✓ Meta title is corrected to include 'The'.
- ✓ Show: full diff of About.tsx.

### Prompt 5 Add Vite manual chunk splitting

#### GOAL

vite.config.ts has no manual chunk configuration. Split large libraries (framer-motion, @tsparticles, react-force-graph-2d, jspdf) into separate chunks to reduce initial bundle size.

#### READ FIRST

vite.config.ts

package.json (dependencies)

src/App.tsx (verify which pages import which heavy libs)

src/pages/Graph.tsx (uses react-force-graph-2d)

#### TASKS

1. Add build.rollupOptions.output.manualChunks to vite.config.ts to split: vendor-react, vendor-motion (framer-motion), vendor-particles (@tsparticles), vendor-graph (react-force-graph-2d), vendor-pdf (jspdf + file-saver), vendor-search (fuse.js).
2. Run `npm run build:compile` and capture chunk size output.
3. Compare total initial JS (non-lazy chunks) before vs after.

#### CONSTRAINTS

- Do NOT change any import statements in page/component files.
- Do NOT change how lazy() imports work in App.tsx.
- Only modify vite.config.ts.

#### ACCEPTANCE CRITERIA

- ✓ `npm run build:compile` succeeds with no new warnings.
- ✓ Build output shows vendor-graph and vendor-particles as separate chunks.
- ✓ Show: full vite.config.ts diff, before/after chunk sizes.

Prompt 6

## Add pre-deploy build validation to GitHub Actions

### GOAL

deploy.yml calls the Netlify hook but never runs `npm run build`. A broken build is only caught by Netlify after the hook fires. Add a build validation step so CI fails fast.

### READ FIRST

`.github/workflows/deploy.yml` (full file)

`netlify.toml`

`package.json` scripts (what does `npm run build` run?)

### TASKS

1. Add a 'Build validation' job step in `deploy.yml` BEFORE the Netlify hook: run `npm run prebuild`, then `npm run build:compile`, then `npm run postbuild` (or just `npm run build`).
2. Ensure the Netlify hook is only called if all prior steps succeed (`if: success()`).
3. Note the double-build tradeoff in a PR comment and recommend approach.

### CONSTRAINTS

- Do not change Netlify build config.
- Do not add extra secrets.
- Be explicit about the double-build tradeoff — document it.

### ACCEPTANCE CRITERIA

- ✓ A PR that breaks `npm run build:compile` shows a red check before Netlify hook fires.
- ✓ `deploy.yml` diff is clean and reviewable.
- ✓ Show: full workflow diff, explanation of approach.

Prompt 7

## Add FTC affiliate disclosure to DisclaimerPage

### GOAL

The `/disclaimer` page contains only a medical disclaimer. Add a compliant affiliate/commercial disclosure section.

### READ FIRST

`src/pages/DisclaimerPage.tsx` (full file)

`src/components/CuratedProductModule.tsx` (existing inline disclosure text)

### TASKS

1. Add a new section after the existing medical disclaimer: `h2` 'Affiliate Disclosure' with text: 'This site participates in affiliate programs including Amazon Associates. When you click certain product links and make a purchase, we may earn a small commission at no extra cost to you. Affiliate links are marked and all product recommendations are independently reviewed before publication.'
2. Render section heading as `<h2>` consistent with page heading hierarchy.

## CONSTRAINTS

- Do not add new components or pages.
- Do not alter the medical disclaimer content.
- Do NOT remove the inline disclosure in CuratedProductModule.

## ACCEPTANCE CRITERIA

- ✓ /disclaimer page contains an 'Affiliate Disclosure' h2 section.
- ✓ Inline disclosure in CuratedProductModule is preserved.
- ✓ Show: full diff of DisclaimerPage.tsx.

### Prompt 8

## Verify prerender coverage matches indexable entity count

### GOAL

Confirm that all indexable herbs (post Prompt 2) are actually included in the prerender plan and generate individual HTML files. Diagnose any gap.

### READ FIRST

```
netlify.toml (PRERENDER_ENTITY_CAP = 2000)
scripts/prerender-static.mjs
scripts/shared-route-manifest.mjs
public/data/publication-manifest.json
scripts/verify-prerender.mjs
```

### TASKS

1. Confirm DEFAULT\_ENTITY\_CAP reads from process.env.PRERENDER\_ENTITY\_CAP and netlify.toml sets it to 2000.
2. Write scripts/smoke-prerender-coverage.mjs that reads publication-manifest.json and the prerender plan, compares them, and reports any indexable herb/compound route missing from the prerender plan with reason.
3. Run the smoke test and report findings.

## CONSTRAINTS

- Do not change PRERENDER\_ENTITY\_CAP (2000 is correct).
- Diagnostic/verification prompt — change as little production code as possible.

## ACCEPTANCE CRITERIA

- ✓ Smoke test runs and produces a clear coverage report.
- ✓ If indexable herbs are missing from prerender plan, root cause is identified.
- ✓ Show: script output, any gaps found.

Prompt 9

## Gate draft blog posts from being committed to main

### GOAL

The daily-post generator creates files with placeholder content and commits them to main via CI. Prevent placeholder content from ever being committed.

### READ FIRST

`scripts/generate-daily-post.mjs` (full file)

`.github/workflows/daily-blog.yml`

`content/blog/` (inspect 2–3 draft `.mdx` files)

`scripts/build-blog.mjs` (confirm draft filtering logic)

### TASKS

1. Write `scripts/validate-blog-post.mjs` that accepts a file path argument and exits 1 with a message if body contains placeholder strings, summary length <20 chars, or empty body.
2. In `generate-daily-post.mjs`, add a check: if body contains 'Placeholder reference A' or 'Placeholder reference B', throw an error before writing the file.
3. Add the validate script call to `daily-blog.yml` BEFORE the git commit step. If it fails, skip commit and log 'Generated post contains placeholder content. Skipping commit.'

### CONSTRAINTS

- Do NOT delete existing draft `.mdx` files (separate cleanup in Prompt 12).
- `build-blog.mjs` draft filter must continue as secondary safety net.
- Do not break `workflow_dispatch` ability.

### ACCEPTANCE CRITERIA

- ✓ Running `generate-daily-post.mjs` with current placeholder template exits non-zero with clear error.
- ✓ `daily-blog.yml` will not commit placeholder content.
- ✓ Show: diffs to `generate-daily-post.mjs` and `daily-blog.yml`.

Prompt 10

## Add BreadcrumbList JSON-LD to herb and compound detail pages

### GOAL

Herb and compound detail pages have `WebPage` JSON-LD but no `BreadcrumbList`. Adding breadcrumbs improves SERP appearance with breadcrumb rich results.

### READ FIRST

`src/lib/seo.ts` (does `breadcrumbJsonLd` already exist?)

`src/pages/HerbDetail.tsx` (how `jsonLd` prop is used)

`src/pages/CompoundDetail.tsx`

`src/components/Meta.tsx`

### TASKS

1. If breadcrumbJsonLd doesn't exist in seo.ts, add it: produces BreadcrumbList schema with position-indexed ListItem array.
2. In HerbDetail.tsx, add breadcrumbJsonLd([Home, Herbs, herb]) to the jsonLd array passed to <Meta>.
3. Do the same for CompoundDetail.tsx with path: Home → Compounds → compound.
4. Run `npm run verify:structured-data` and confirm no new errors.

#### CONSTRAINTS

- Do not change how Meta component renders JSON-LD.
- No visible UI breadcrumb elements — schema only.

#### ACCEPTANCE CRITERIA

- ✓ Both HerbDetail and CompoundDetail include BreadcrumbList in JSON-LD.
- ✓ `npm run verify:structured-data` exits 0.
- ✓ Show: diffs to seo.ts (if changed), HerbDetail.tsx, CompoundDetail.tsx.

Prompt  
11

## Add Article JSON-LD to blog posts

#### GOAL

Blog post pages should have Article JSON-LD (author, datePublished, dateModified, image) to qualify for Article rich results in SERPs.

#### READ FIRST

`src/lib/seo.ts` (blogJsonLd function — verify it produces Article schema)  
`src/pages/BlogPost.tsx` (does it pass jsonLd to <Meta>?)  
`src/data/blog/posts.json` (sample post structure)

#### TASKS

1. Verify BlogPost.tsx passes jsonLd to <Meta>. If not, add the call.
2. Verify blogJsonLd() produces @type: 'Article' with author, datePublished, dateModified, headline, and image. Add any missing fields.
3. Ensure author is a Person object: { '@type': 'Person', 'name': 'The Hippie Scientist' }.

#### CONSTRAINTS

- Only modify seo.ts and BlogPost.tsx.
- Do not change the blog data schema.

#### ACCEPTANCE CRITERIA

- ✓ blogJsonLd() output includes: @type Article, author Person, datePublished, headline, image.
- ✓ Show: diff of seo.ts and BlogPost.tsx.

Prompt  
12

## Clean up the 60 draft daily-note MDX files

### GOAL

Remove the 60 draft: true daily-note MDX files from content/blog/ and move them to an archive folder to stop repo bloat.

### READ FIRST

content/blog/ (list all files)

scripts/build-blog.mjs (confirm draft filter — these are already excluded)

src/data/blog/posts.json (confirm none of the draft slugs appear here)

### TASKS

1. Write scripts/clean-draft-posts.mjs that: scans content/blog/ for files with draft: true AND 'Placeholder reference A' in body; lists them (--dry-run default); with --apply, moves them to .draft-archive/ folder at repo root.
2. Run with --dry-run first and report candidate list.
3. Run with --apply to move the files.
4. Add .draft-archive/ to .gitignore.
5. Confirm `npm run build:blog` still exits 0 with same published post count.

### CONSTRAINTS

- Do NOT git rm files — move to .draft-archive/ to preserve history.
- Do NOT remove any file that appears in src/data/blog/posts.json.

### ACCEPTANCE CRITERIA

- ✓ content/blog/ contains no files with 'Placeholder reference A' in body.
- ✓ `npm run build:blog` exits 0 with same published post count.
- ✓ Show: dry-run output, apply output, final post count.

Prompt  
13

## Improve homepage CTA hierarchy

### GOAL

The homepage has 8+ sections with no single primary action foregrounded. Improve conversion clarity without a full redesign.

### READ FIRST

src/pages/Home.tsx (full file)

src/components/Hero.tsx

src/components/EmailCapture.tsx

### TASKS

1. Inspect the Hero component's primary CTA button text. If generic (e.g. 'Learn more'), change to 'Explore the Herb Database' or 'Find herbs by effect'.

2. Change the EmailCapture section label from 'Newsletter' to a benefit-oriented headline like 'Research notes in your inbox' with a subtitle: 'New herb profiles, safety updates, and mechanism breakdowns — no wellness fluff.'
3. Verify the homepage Meta description is unchanged (current description is good).

#### CONSTRAINTS

- No new components.
- No layout restructuring — copy/label improvements only.
- Keep all existing sections.

#### ACCEPTANCE CRITERIA

- ✓ Hero CTA is specific and action-oriented.
- ✓ EmailCapture has a benefit-oriented label.
- ✓ Show: diff of Home.tsx and Hero.tsx.

Prompt  
14

### Add RSS autodiscovery link to index.html

#### GOAL

The site generates /feed.xml but the HTML <head> does not advertise it via <link rel='alternate'>. Feed readers and Google cannot auto-discover it.

#### READ FIRST

index.html (the Vite entry HTML template)

scripts/prerender-static.mjs (does it strip link rel='alternate' tags? — check HEAD\_REPLACE\_PATTERNS)

public/feed.xml (confirm it exists)

#### TASKS

1. Add to index.html <head>: `<link rel='alternate' type='application/rss+xml' title='The Hippie Scientist – Research Notebook' href='/feed.xml' />`
2. Confirm prerender-static.mjs does NOT strip this tag (HEAD\_REPLACE\_PATTERNS only targets title, canonical, description, og:\*, twitter:\* — not link rel='alternate').
3. After build, verify: `grep -r 'alternate.*rss' dist/index.html` returns the tag.

#### CONSTRAINTS

- Only modify index.html.
- Do not add a second link for /rss.xml — one canonical feed URL is sufficient.

#### ACCEPTANCE CRITERIA

- ✓ index.html contains the RSS autodiscovery link.
- ✓ `grep -r 'alternate.*rss' dist/index.html` returns the tag after build.
- ✓ Show: diff of index.html.



Prompt  
15

## Re-enable targeted ESLint rules as warnings

### GOAL

Re-enable no-unused-vars and react-hooks/exhaustive-deps as 'warn' to make violations visible without blocking CI, and report what currently violates them.

### READ FIRST

`eslint.config.js` (full file)

`package.json` (the lint script uses `--max-warnings=0`)

### TASKS

1. Change `@typescript-eslint/no-unused-vars` from 'off' to 'warn'.
2. Change `react-hooks/exhaustive-deps` from 'off' to 'warn'.
3. Leave `no-explicit-any` as 'off' for now (too many to fix at once).
4. Change the lint script to `--max-warnings=50` temporarily with a TODO comment.
5. Run `npm run lint` and report: total warnings, files/rules with most violations.
6. Do NOT fix the violations in this PR — only report them.

### CONSTRAINTS

- Do not fix any lint violations in this PR.
- Do not remove any currently-enabled rules.

### ACCEPTANCE CRITERIA

- ✓ `npm run lint` exits 0 with new `--max-warnings=50` threshold.
- ✓ PR description includes: 'X unused-var warnings, Y exhaustive-deps warnings, top 5 files by violation count.'
- ✓ Show: diff of `eslint.config.js`, sample of lint output.

Prompt  
16

## Add robots.txt and verify sitemap reference

### GOAL

Verify a proper `robots.txt` exists in `/public`, is served from the root, allows crawling of all entity pages, and references the sitemap.

### READ FIRST

`public/robots.txt` (does it exist? what does it say?)

`scripts/generate-sitemap.mjs` (what path does it write the sitemap to?)

`netlify.toml`

### TASKS

1. Create/fix `public/robots.txt`: User-agent: \*, Allow: /, Disallow: /analytics /data-fix /graph /theme /preview, Sitemap: <https://thehippiescientist.net/sitemap.xml>
2. Confirm the sitemap is generated to `dist/sitemap.xml` after build.

3. Add a smoke test to scripts/verify-publishing.mjs (or a new script) that reads dist/robots.txt and exits 1 if it's missing 'Allow: /' or 'Sitemap:'.

#### CONSTRAINTS

- Do not disallow any herb, compound, blog, or collection routes.
- /analytics disallow is correct — that route is gated to internal use.

#### ACCEPTANCE CRITERIA

- ✓ public/robots.txt exists with correct content.
- ✓ `grep -i sitemap dist/robots.txt` after build returns the sitemap URL.
- ✓ Show: robots.txt content, any diff.

Prompt  
17

### Audit and fix internal linking on herb detail pages

#### GOAL

Herb detail pages should link to related compounds, collection pages, and blog posts. Audit what currently exists and add missing cross-links.

#### READ FIRST

`src/pages/HerbDetail.tsx` (full file)

`src/lib/discovery.ts`

`src/data/seoCollections.ts`

`src/lib/blog.ts`

#### TASKS

1. Audit: does HerbDetail currently render links to (a) related compound pages, (b) related collection pages, (c) related blog posts? Print findings.
2. For any absent category, add a 'Related' section at page bottom (max 5 items per category, only show if  $\geq 1$  item, use existing discovery.ts utilities).
3. Ensure all new links use `<Link to='...'>` not `<a href='...'>`.

#### CONSTRAINTS

- Do not change main herb profile content sections.
- New sections go below existing content.
- Use existing UI components for consistency.

#### ACCEPTANCE CRITERIA

- ✓ At least 2 of 3 link categories present on a herb page with known associations.
- ✓ All links are `<Link>` not `<a>`.
- ✓ Show: diff of HerbDetail.tsx.

Prompt  
18

## Add meta author tag and strengthen organization JSON-LD

### GOAL

Add `<meta name='author'>` to prerendered pages and improve `organizationJsonLd` with a `contactPoint` to strengthen E-E-A-T signals.

### READ FIRST

`src/components/Meta.tsx`

`src/lib/seo.ts` (`organizationJsonLd`)

`scripts/prerender-static.mjs` (`HEAD_REPLACE_PATTERNS` — confirm author is not stripped)

### TASKS

1. In `src/components/Meta.tsx`, add `<meta name='author' content='The Hippie Scientist' />` to the rendered head tags.
2. In `src/lib/seo.ts`, update `organizationJsonLd` to include `contactPoint`: `{ @type: ContactPoint, contactType: 'editorial', url: 'https://thehippiescientist.net/contact' }`.
3. Confirm `prerender-static.mjs` does NOT strip `meta name='author'` (check `HEAD_REPLACE_PATTERNS`).

### CONSTRAINTS

- Do not add fake social URLs. If no profiles are established, omit `sameAs`.
- Meta/schema addition only — no UI changes.

### ACCEPTANCE CRITERIA

- ✓ After build, `grep 'name="author"' dist/herbs/*/index.html` returns the meta tag in at least one prerendered herb page.
- ✓ `organizationJsonLd` output includes `contactPoint`.
- ✓ Show: diffs to `Meta.tsx` and `seo.ts`.

Prompt  
19

## Add word count and reading time to blog posts

### GOAL

Blog posts show no reading time estimate. Add `wordCount` and `readingTimeMinutes` to the build pipeline and display them on post pages.

### READ FIRST

`scripts/build-blog.mjs`

`src/types/blog.ts`

`src/pages/BlogPost.tsx`

### TASKS

1. In `build-blog.mjs`, after extracting post body, compute: `wordCount` (plain-text word count, strip HTML) and `readingTimeMinutes = Math.ceil(wordCount / 200)`. Write both to `posts.json` entries.

2. Update `src/types/blog.ts`: add `wordCount?: number` and `readingTimeMinutes?: number`.
3. In `BlogPost.tsx`, display `'{readingTimeMinutes} min read'` near the post date/meta line (only if `readingTimeMinutes > 0`).

#### CONSTRAINTS

- Both new fields are optional (?) in the type.
- Do not add a new npm package — plain JS word count is sufficient.

#### ACCEPTANCE CRITERIA

- ✓ `npm run build:blog` populates `wordCount` and `readingTimeMinutes` in `posts.json`.
- ✓ Long-form posts (e.g. `blue-lotus-aporphines`) show at least 4 min read.
- ✓ Show: diff of `build-blog.mjs`, `blog.ts`, `BlogPost.tsx`, sample `posts.json` entry.

Prompt  
20

### Instrument key events in Blend Builder and Interaction Checker

#### GOAL

Audit current analytics event coverage and add `blend_generated`, `interaction_check_run`, and `affiliate_click` events to the most conversion-valuable user actions.

#### READ FIRST

`src/lib/useGA.ts`  
`src/lib/contentJourneyTracking.ts`  
`src/pages/BuildBlend.tsx`  
`src/pages/InteractionsPage.tsx`  
`src/components/CuratedProductModule.tsx`

#### TASKS

1. Audit: list every `gtag` / `trackEvent` / `analytics` call in the codebase. Print event names and locations.
2. Add `blend_generated` event in `BuildBlend.tsx` when a blend is created: `{ herb_count, intent }`.
3. Add `interaction_check_run` event in `InteractionsPage.tsx` when check is submitted: `{ herb_count, warning_count }`.
4. Add `affiliate_click` event in `CuratedProductModule.tsx` when a product link is clicked: `{ product_name, herb_slug, position }`.
5. Use existing analytics utility — do NOT add a new tracking library.

#### CONSTRAINTS

- Respect the existing consent gate — do not fire events before consent.
- Do not add Google Analytics or any new script tags.
- Do not remove or rename existing events.

#### ACCEPTANCE CRITERIA

- ✓ The three new events fire on the correct actions (verified via browser console in dev mode).

✓ Show: diffs to BuildBlend.tsx, InteractionsPage.tsx, CuratedProductModule.tsx.



## 5. Recommended Prompt Order

---

### Wave 1 — Data Integrity

Run before any SEO work. Data corruption blocks everything downstream.

- Prompt 1 — NaN scrub (must go first; corrupts everything downstream)
- Prompt 2 — Herb descriptions (directly unlocks indexable pages)
- Prompt 3 — Compound descriptions (same)
- Prompt 8 — Prerender coverage audit (verify Wave 1 actually works end-to-end)

### Wave 2 — Trust & Credibility

Easy wins, low risk, high perceived impact.

- Prompt 4 — About page fix (one-line change, high trust impact)
- Prompt 7 — Affiliate disclosure on Disclaimer (compliance value)
- Prompt 16 — robots.txt (quick, foundational)

### Wave 3 — SEO Signals

Build on clean data from Wave 1.

- Prompt 10 — BreadcrumbList JSON-LD
- Prompt 11 — Article JSON-LD on blog
- Prompt 14 — RSS autodiscovery link
- Prompt 18 — Author meta + org JSON-LD

### Wave 4 — Performance & Build Quality

Compound savings over time.

- Prompt 5 — Vite chunk splitting
- Prompt 6 — CI build validation
- Prompt 15 — ESLint re-enable rules

### Wave 5 — Content & UX

Polish and conversion once foundation is solid.

- Prompt 9 — Gate draft blog posts
- Prompt 12 — Clean draft files
- Prompt 13 — Homepage CTA hierarchy
- Prompt 17 — Internal linking on herb pages
- Prompt 19 — Reading time



## 6. PR Review Checklist

---

Use this checklist to decide whether a Codex PR is real progress or fake progress.

### Does it do what it says?

- The PR title matches the Codex prompt title exactly
- The diff is scoped to files mentioned in the prompt; no unrelated changes
- The acceptance criteria from the prompt are explicitly verified in the PR description

### Data safety

- No changes to raw data files in `src/data/herbs/` or `src/data/compounds/` (those are locked)
- Any changes to `public/data/*.json` are produced by a script, not handwritten
- No 'nan' string literals appear as new values in data files

### Build safety

- `npm run build` passes locally (or CI shows green)
- `npm run verify:publishing` passes
- `npm run verify:prerender` passes (if entity data was touched)

### SEO safety

- No existing URL patterns are changed without a corresponding redirect
- No `noindex` is added to pages that should be indexed
- Canonical tags are not broken

### Performance check

- If a new import was added to a non-lazy page, bundle impact is noted
- No synchronous fetch or heavy computation added to the critical render path

### Fake progress signals to REJECT

- PR does not just add comments without fixing the underlying issue
- PR does not lower quality thresholds to make more pages 'pass' without improving data
- PR does not disable lint rules to make lint pass
- PR does not raise `--max-warnings` without a corresponding list of what was fixed



## 7. Optional Stretch Prompts

High-value but not core. Run these after the 20 primary prompts are complete.

### Stretch A Real LLM-powered daily post generation

Replace the static template in `generate-daily-post.mjs` with a real Anthropic or OpenAI API call that generates 400–600 word herb mechanism posts using the herb's data from `herbs_combined_updated.json`. Include: mechanism summary, traditional use context, safety notes, and 2–3 cited sources from the herb's `sources` array. Gate behind `ANTHROPIC_API_KEY` secret in CI. Mark posts `draft: false` only if the API call succeeds and word count is  $\geq 300$ .

### Stretch B LLM batch enrichment for compound descriptions

Write an enrichment script that takes the top 100 compounds by name recognizability (filter out pure IUPAC names) and calls an LLM API to generate 100–150 word pharmacology summaries. Validate output: no fabricated drug claims, must include mechanism mention, must mention at least one herb association. Write to the existing `enrichment-governed.json` format.

### Stretch C Bundle size regression CI check

Add a GitHub Actions step that after `npm run build:compile` reads `dist/` chunk sizes, compares against a baseline stored in `.bundle-baseline.json`, and fails if any chunk grows by more than 20%. Catches accidental heavy imports before they ship.

### Stretch D Preload critical data files in prerendered HTML

Herb and compound pages fetch their data from `public/data` JSON files. Add `<link rel='preload' as='fetch'>` tags in the prerendered HTML for each entity page's corresponding detail JSON file to reduce the fetch waterfall depth.

### Stretch E Sitemap index + per-category sitemaps

When herb+compound indexable counts exceed 500, the single `sitemap.xml` will get large. Add a sitemap index file at `/sitemap.xml` that references: `/sitemap-core.xml`, `/sitemap-herbs.xml`, `/sitemap-compounds.xml`, `/sitemap-blog.xml`. Each sub-sitemap generated by a dedicated function in `generate-sitemap.mjs`.

### Stretch F Conversion scorecard editorial dashboard

The `report:conversion-scorecard` script exists. Wire it into a lightweight internal `/analytics` page section showing: affiliate-eligible herb pages (high confidence + curated product available), pages missing affiliate slots, and affiliate click-through rates from GA events. Turns existing scripts into an actionable editorial dashboard.